

Memory Management Simulator

A visual, interactive simulator for OS memory management concepts — built to go beyond textbook diagrams and actually see what happens inside RAM.

Live Demo → [Memory Management Simulator](#)

Why I Built This

In my OS lab, I kept writing C++ programs that output page fault counts in a terminal — numbers with no visual context. I understood the algorithms on paper but couldn't see what was actually happening inside RAM at each step.

So I built this — a simulator where you can watch pages load and evict in real time, compare algorithms side by side, and even catch Belady's anomaly happening automatically.

What It Simulates

Page Replacement Algorithms

Algorithm Strategy		Best For
FIFO	Evict the oldest page	Simple, but suffers Belady's anomaly
LRU	Evict least recently used	Best practical performance
Optimal	Evict page not needed longest	Theoretical best (needs future knowledge)
Clock	Second chance before eviction	What real OS actually uses

TLB — Translation Lookaside Buffer

A hardware cache that sits in front of the page table. The simulator shows:

- Which pages are currently in TLB
- TLB hit vs miss on every memory access
- Hit rate and effective access time impact

Belady's Anomaly Detection

Automatically detects when FIFO produces *more* page faults with *more* frames — a counterintuitive phenomenon that doesn't occur in LRU or Optimal.

Hardest Parts To Build

LRU Implementation Maintaining page access order efficiently was the trickiest part. A naive approach using arrays is $O(n)$ per access. The proper solution uses a doubly linked list + hashmap — same pattern as **LeetCode #146** — achieving **$O(1)$** for both lookup and eviction.

TLB Layer Adding TLB on top of page replacement means two levels of lookup per memory access. Getting the hit/miss logic right across all four algorithms without breaking existing behaviour took careful testing.

Belady's Anomaly Detection Running FIFO simulation across multiple frame counts ($n, n+1, n+2\dots$) and comparing results automatically — then highlighting the exact point where more frames caused more faults.

Clock Algorithm The circular buffer with reference bits and the "second chance" sweep logic was unintuitive at first. Once it clicked, I understood why Linux actually uses a variant of this over pure LRU.

Features

- **Step-by-step mode** — go through one memory access at a time
 - **Algorithm comparison** — run all 4 algorithms on same input simultaneously
 - **Belady's anomaly** — auto-detected and highlighted with explanation
 - **TLB simulation** — hit/miss tracking with hit rate stats
 - **Timeline table** — full page-by-page trace like OS textbooks show
 - **Random reference string** — generate random inputs instantly
-

Tech Stack

HTML / CSS / JavaScript — no frameworks, no libraries

Pure vanilla JS so the logic is transparent and readable

Keeping it framework-free was intentional — the focus is on the algorithms, not the tooling.

How To Run Locally

```
git clone https://github.com/harshita1262/memory-management-simulator.git
cd memory-management-simulator
```

No build step. No npm install. Just open and run.

What I Learned

- LRU cache is not just an OS concept — it's a core DSA pattern (doubly linked list + hashmap) that shows up in system design interviews
 - Belady's anomaly is real and easy to trigger — FIFO is not a stack algorithm, which is why modern OS don't use it
 - TLB hit rate has a massive impact on effective memory access time — even a 90% hit rate reduces average access time dramatically
 - Clock algorithm is elegant — approximating LRU without the overhead, which is why Linux uses a variant of it
-

Roadmap

- [] Working Set Model implementation
 - [] Thrashing detection with graph
 - [] Segmentation simulation
 - [] Multi-level page table visualizer
 - [] Export simulation results as PDF
-

Connect

Built by **Harshita** — CSE NSUT'28

[Harshita Goyal](#) | [LinkedIn](#) | [harshita1262](https://github.com/harshita1262)

Always open to feedback and suggestions.